

HIX UI Automation Testing Framework - Comprehensive Documentation

Version: 1.0

Last Updated: December 2024

Framework: Playwright + Cucumber.js

Project: Health Insurance Exchange (HIX) UI Automation Tests

Table of Contents

- [1. Executive Summary](#)
 - [2. Framework Overview](#)
 - [3. Architecture & Design Patterns](#)
 - [4. Technology Stack](#)
 - [5. Project Structure](#)
 - [6. Core Components Deep Dive](#)
 - [7. State Management System](#)
 - [8. Selector & Content Management](#)
 - [9. Test Execution Flow](#)
 - [10. Data Management](#)
 - [11. Reporting & Analytics](#)
 - [12. Best Practices & Guidelines](#)
 - [13. Troubleshooting Guide](#)
 - [14. Advanced Topics](#)
-

1. Executive Summary

1.1 What is This Framework?

The HIX UI Automation Testing Framework is a **comprehensive, enterprise-grade test automation solution** designed specifically for testing Health Insurance Exchange (HIX) platforms across multiple US states. Built on **Playwright** and **Cucumber.js**, it provides a robust, maintainable, and scalable approach to UI testing.

1.2 Key Capabilities

- Multi-State Testing:** Supports 9+ US states (PA, NJ, NV, ID, GA, VA, MN, IL, NM) with state-specific configurations
- BDD Framework:** Behavior-Driven Development using Gherkin syntax for readable, maintainable tests
- Page Object Model:** Encapsulated page interactions for better maintainability
- Centralized Selector Management:** State-aware selector resolution system
- Content Management:** Multi-language and state-specific content handling
- Comprehensive Reporting:** Allure integration with detailed test analytics
- API Integration:** Combined UI and API testing capabilities
- Database Integration:** Direct database validation and test data setup
- CI/CD Ready:** Jenkins pipeline integration for continuous testing

1.3 Framework Statistics

- Test Scenarios:** 1,900+ feature files
- Page Models:** 300+ page object classes

- **Step Definitions:** 260+ step definition files
 - **Supported States:** 9+ US states
 - **Test Coverage:** SSAP, Enrollment, Member Portal, Agency Management, API Testing, Accessibility
-

2. Framework Overview

2.1 Framework Philosophy

The framework follows these core principles:

1. **Maintainability:** Centralized selector and content management reduces maintenance overhead
2. **Scalability:** Modular architecture supports easy addition of new states and features
3. **Reusability:** Common utilities and page models promote code reuse
4. **Readability:** BDD approach makes tests readable by non-technical stakeholders
5. **Reliability:** Robust error handling and retry mechanisms ensure stable test execution

2.2 Testing Approach

The framework uses a **hybrid testing approach**:

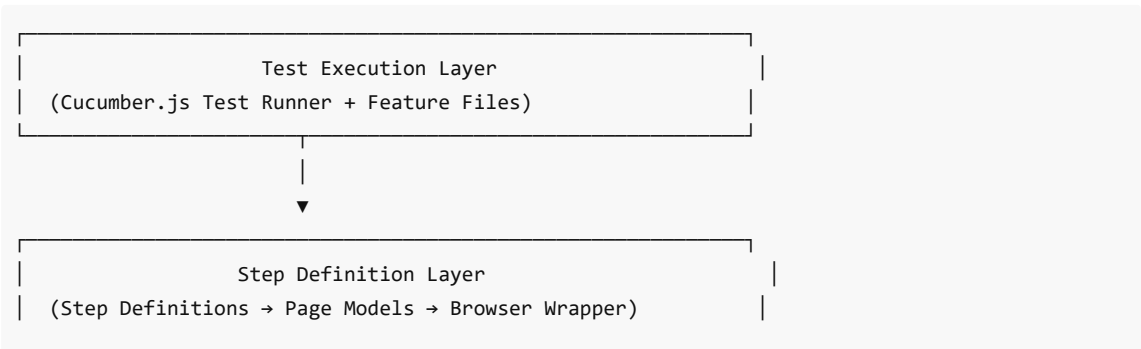
- **BDD (Behavior-Driven Development):** Tests written in Gherkin syntax
- **Page Object Model (POM):** UI interactions abstracted into page classes
- **Data-Driven Testing:** External test data files for parameterized testing
- **Keyword-Driven:** Reusable step definitions act as keywords
- **API-UI Integration:** Combined API and UI validation

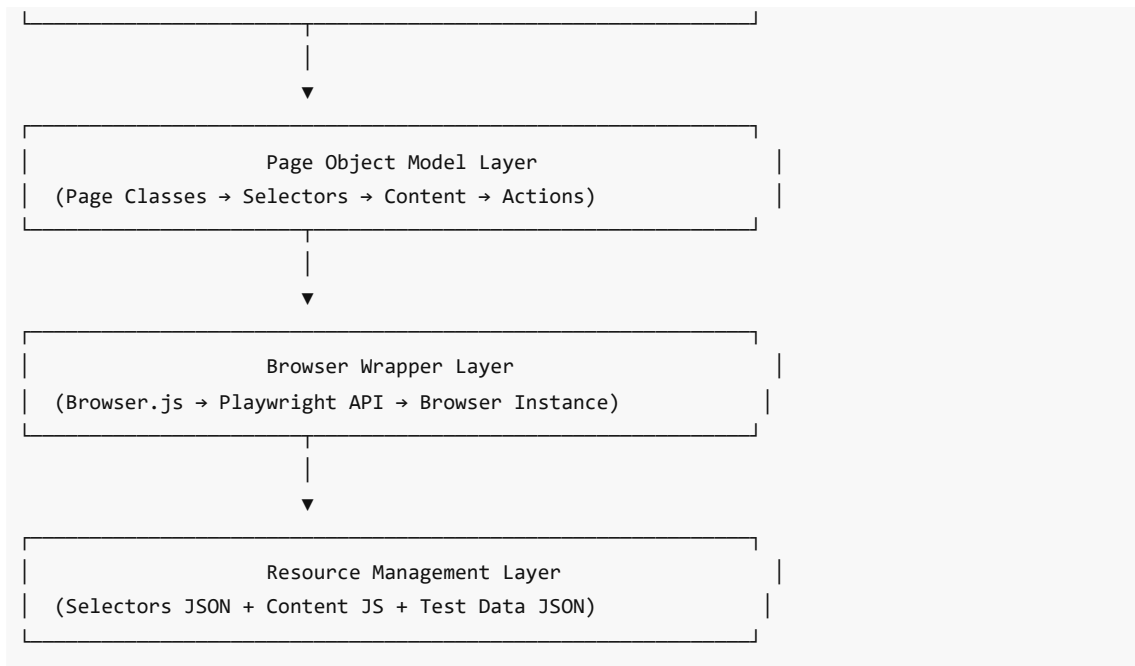
2.3 Supported Application Areas

1. **SSAP (Single Streamlined Application Process):** Core application functionality
 2. **Account Management:** User registration, login, profile management
 3. **Plan Display:** Insurance plan browsing and comparison
 4. **Enrollment:** Insurance plan enrollment and management
 5. **Agency/Agent/Broker Management:** B2B functionality
 6. **Member Portal:** Customer self-service portal
 7. **API Testing:** Backend service validation
 8. **Accessibility Testing:** WCAG compliance verification
 9. **Renewal Processing:** Annual renewal workflows
 10. **Report A Change:** Life event change processing
-

3. Architecture & Design Patterns

3.1 High-Level Architecture





3.2 Design Patterns Used

3.2.1 Page Object Model (POM)

Purpose: Encapsulate page-specific elements and actions

Implementation:

```
class LoginPage {
  constructor() {
    // Load selectors from JSON file
    const selectors = require('../../resources/selectors/common/SSAP/LoginPage.json');
    this.emailField = selectors.emailField;
    this.passwordField = selectors.passwordField;
    this.submitButton = selectors.submitButton;
  }

  async login(email, password) {
    await browser.setValueInTextField(this.emailField, email);
    await browser.setValueInTextField(this.passwordField, password);
    await browser.click(this.submitButton);
  }
}
```

Benefits:

- Separation of test logic from page implementation
- Reusable page interactions
- Easy maintenance when UI changes

3.2.2 Factory Pattern

Purpose: Create browser instances based on configuration

Implementation:

```
// In Hooks.js
if (global.browserOption.toLowerCase() === 'firefox') {
  global.pwBrowser = await firefox.launch({headless: true});
} else {
  global.pwBrowser = await chromium.launch({headless: true});
}
```

3.2.3 Strategy Pattern

Purpose: State-specific selector and content resolution

Implementation:

```
// Browser.js - getSelector method
getSelector(selector) {
  if (typeof selector !== "string") {
    if (selector[state.toUpperCase()]) {
      return selector[state]; // State-specific
    } else {
      return selector.common; // Fallback to common
    }
  }
  return selector; // String selector
}
```

3.2.4 Singleton Pattern

Purpose: Global browser instance management

Implementation:

```
// Global browser instance
global.browser = new Browser(global.page);
```

3.2.5 Data-Driven Testing Pattern

Purpose: Parameterized tests with external data

Implementation:

```
Scenario Outline: Test with multiple data sets
  Given I have test data "<dataSet>"
  When I perform the action
  Then I should see expected result
  Examples:
    | dataSet |
    | validData1 |
    | validData2 |
```

3.3 Framework Layers

Layer 1: Test Specification (Feature Files)

- **Location:** tests/features/
- **Format:** Gherkin syntax (.feature files)
- **Purpose:** Define test scenarios in natural language

Layer 2: Test Implementation (Step Definitions)

- **Location:** tests/stepDefs/
- **Format:** JavaScript (.js files)
- **Purpose:** Implement test logic for each Gherkin step

Layer 3: Page Interaction (Page Models)

- **Location:** tests/pagemodels/
- **Format:** JavaScript classes (.js files)
- **Purpose:** Encapsulate page-specific interactions

Layer 4: Browser Abstraction (Browser Wrapper)

- **Location:** tests/base/Browser.js
- **Format:** JavaScript class
- **Purpose:** Abstract Playwright APIs with enhanced functionality

Layer 5: Resource Management (Selectors & Content)

- **Location:** resources/selectors/ and resources/content/
 - **Format:** JSON and JavaScript modules
 - **Purpose:** Centralized UI element and content management
-

4. Technology Stack

4.1 Core Technologies

Technology	Version	Purpose
Node.js	18.0+	Runtime environment
Playwright	1.56.1	Browser automation
Cucumber.js	10.9.0	BDD test framework
JavaScript	ES6+	Programming language

4.2 Key Dependencies

Testing Framework

- @cucumber/cucumber : BDD framework
- @playwright/test : Browser automation
- chai : Assertion library

Reporting

- allure-cucumberjs : Allure integration for Cucumber
- allure-commandline : Allure report generation

- `allure-js-commons` : Core Allure library

Utilities

- `axios` : HTTP client for API testing
- `lodash` : Utility functions
- `moment` : Date manipulation
- `jsonpath` : JSON querying
- `xlsx` : Excel file processing

Database

- `oracledb` : Oracle database connectivity
- `pg-promise` : PostgreSQL operations

Other

- `dotenv` : Environment configuration
- `env-cmd` : Environment variable management
- `cross-env` : Cross-platform environment variables

4.3 Supporting Tools

- **Jenkins**: CI/CD pipeline
- **Docker**: Containerization (optional)
- **Git**: Version control
- **Allure**: Test reporting and analytics
- **Java 8+**: Required for Allure CLI

5. Project Structure

5.1 Directory Structure

```
ghix_ui_automation_tests/
├── config/                                # Configuration files
│   ├── testConfig.properties             # Test configuration
│   ├── db.properties                     # Database configuration
│   └── ServerHealthEndpoints.json        # Health check endpoints
├── tests/                                # Main test directory
│   ├── features/                         # Gherkin feature files
│   │   ├── SSAP/                        # Single Streamlined Application Process
│   │   ├── Eligibility/                 # Eligibility testing
│   │   ├── MemberPortal/                # Member portal tests
│   │   ├── Agency/                      # Agency management
│   │   ├── API/                         # API testing scenarios
│   │   └── [other modules]/              # Additional test areas
│   ├── stepDefs/                         # Step definition implementations
│   │   └── [mirrors features/]           # Same structure as features/
│   ├── pagemodels/                       # Page Object Models
│   │   ├── SSAP/                        # SSAP page models
│   │   └── MemberPortal/                 # Member portal pages
```

```

| | | └─ [other modules]/          # Additional page models
| | |
| | └─ base/                      # Base classes
| |   └─ Browser.js              # Browser wrapper class
| |   └─ Assert.js              # Assertion utilities
| |
| | └─ common.utils/              # Common utilities
| |   └─ Hooks.js                # Cucumber hooks
| |   └─ Constants.js            # Framework constants
| |   └─ DataUtil.js             # Data utilities
| |   └─ LoggerUtil.js           # Logging utilities
| |   └─ [other utilities]/      # Additional utilities
| |
| └─ resources/                  # Test resources
|   └─ data/                    # Test data files
|     └─ Common/                # Shared test data
|     └─ PA/                    # Pennsylvania-specific data
|     └─ NJ/                    # New Jersey-specific data
|     └─ [other states]/        # State-specific data
|   └─ selectors/               # UI element selectors
|     └─ common/                # Shared selectors
|       └─ SSAP/                # SSAP selectors
|       └─ MemberPortal/        # Member portal selectors
|       └─ [other modules]/      # Additional selectors
|     └─ exchange/              # State-specific selectors
|       └─ PA/                  # Pennsylvania selectors
|       └─ NJ/                  # New Jersey selectors
|       └─ [other states]/       # State-specific overrides
|   └─ content/                 # UI content/text
|     └─ common/                # Shared content
|     └─ exchange/              # State-specific content
|   └─ payloads/                # API request payloads
|     └─ Common/                # Shared payloads
|     └─ [state folders]/       # State-specific payloads
|
| └─ allure-results/             # Allure test results (generated)
| └─ allure-report/              # Allure HTML reports (generated)
| └─ reports/                    # Test execution reports
| └─ screenshot/                 # Test failure screenshots
|
| └─ package.json                # Dependencies and scripts
| └─ playwright.config.js        # Playwright configuration
| └─ cucumber.js                 # Cucumber test runner config
| └─ environment.env             # Environment variables
| └─ README.md                   # Project documentation

```

5.2 Key Configuration Files

5.2.1 package.json

Defines project dependencies and npm scripts:

```
{
  "scripts": {
    "test": "env-cmd -f ./environment.env cucumber-js test",
    "allure:generate": "npx allure generate ./allure-results --clean",
    "allure:open": "allure open allure-report"
  }
}
```

5.2.2 cucumber.js

Cucumber test runner configuration:

```
const common = `
  --require ./tests/common.utils/Hooks.js
  --require ./tests/stepDefs/**/*.js
  --format ./tests/common.utils/reporter.js
  --parallel 16
  --retry 1
`;
```

5.2.3 playwright.config.js

Playwright browser configuration (used for browser setup, not test execution):

```
module.exports = defineConfig({
  testDir: './tests',
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } }
  ]
});
```

5.2.4 environment.env

Environment-specific variables:

```
BROWSER=chromium
state=PA
applicationYear=2025
SERVER=pa2auto.eng.vimo.com
tagExpression=@smoke
```

6. Core Components Deep Dive

6.1 Browser Wrapper (Browser.js)

The Browser wrapper class provides an abstraction layer over Playwright APIs with enhanced functionality.

Key Features

1. State-Aware Selector Resolution

```
getSelector(selector) {
  if (typeof selector !== "string") {
    if (selector[state.toUpperCase()]) {
      return selector[state]; // State-specific
    } else {
      return selector.common; // Fallback
    }
  }
  return selector;
}
```

2. Content Resolution

```
getContent(content) {
  if (typeof content !== "string") {
    if (global.uiLanguage === "ES") {
      // Spanish content resolution
      return content.spanish[state] || content.spanish.common;
    } else {
      // English content resolution
      return content[state] || content.common;
    }
  }
  return content;
}
```

3. Enhanced Waiting Mechanisms

```
async waitForLoadingSpinnerToHide() {
  // Waits for multiple loader types
  // Handles state transitions
  // Provides timeout handling
}
```

4. Element Interaction Methods

- `click(selector)` : Click element with wait
- `setValueInTextField(selector, value)` : Set text field value
- `getText(selector)` : Get element text
- `waitForElementToDisplay(selector)` : Wait for element visibility
- `waitUntilElementIsClickable(selector)` : Wait for clickability

Browser Wrapper Methods

Method	Purpose
--------	---------

<code>getSelector(selector)</code>	Resolve state-specific selector
<code>getContent(content)</code>	Resolve state/language-specific content
<code>click(selector)</code>	Click element with proper waits
<code>setValueInTextField(selector, value)</code>	Set text field value
<code>getText(selector)</code>	Get element text content
<code>waitForElementToDisplay(selector)</code>	Wait for element visibility
<code>waitForLoadingSpinnerToHide()</code>	Wait for loaders to disappear
<code>takeScreenshot()</code>	Capture page screenshot
<code>navigateToGivenUrl(url)</code>	Navigate to URL
<code>waitForPageLoad(selector, content)</code>	Wait for page to load

6.2 Hooks (Hooks.js)

Cucumber hooks manage test lifecycle events.

BeforeAll Hook

- Initializes global variables
- Launches browser instance
- Sets up environment configuration
- Initializes database connections

Before Hook (Per Scenario)

- Creates new browser context
- Sets up test data
- Performs pre-test setup

After Hook (Per Scenario)

- Cleans up test data
- Closes browser context
- Handles test teardown

AfterStep Hook

- Captures screenshots on failure
- Attaches logs to Allure report
- Handles soft assertions

6.3 Page Models

Page models encapsulate page-specific interactions.

Structure

```
class PageName {
  constructor() {
    // Load selectors
  }
}
```

```

const selectors = require('../../resources/selectors/common/Module/PageName.json');
this.elementName = selectors.elementName;

// Load content
const content =
require('../../resources/content/common/Module/PageName.content.js');
this.expectedText = content.expectedText;
}

// Action methods
async performAction() {
  const selector = browser.getSelector(this.elementName);
  await browser.click(selector);
}

// Verification methods
async verifyPage() {
  const expectedText = browser.getContent(this.expectedText);
  const actualText = await browser.getText(this.elementName);
  expect(actualText).toContain(expectedText);
}
}

```

6.4 Step Definitions

Step definitions implement Gherkin steps.

Structure

```

const { Given, When, Then } = require('@cucumber/cucumber');
const PageModel = require('../../pagemodels/Module/PageModel');

Given('I am on the {string} page', async function(pageName) {
  await browser.navigateToGivenUrl(pageUrl);
  await pageModel.waitForPageToLoad();
});

When('I click the {string} button', async function(buttonName) {
  await pageModel.clickButton(buttonName);
});

Then('I should see {string}', async function(expectedText) {
  await pageModel.verifyText(expectedText);
});

```

6.5 Global Data Store

The framework uses a global data store (`global.updateDataJson`) to share data across test steps.

Structure

```
global.updateDataJson = {
  households: [],      // Household member data
  agencies: [],        // Agency information
  entities: [],         // Entity data
  users: [],           // User account data
  enrollment: [],       // Enrollment records
  sessionId: "",        // Current session ID
  api: {}              // API response data
};
```

7. State Management System

7.1 Multi-State Architecture

The framework supports testing across multiple US states, each with:

- State-specific UI selectors
- State-specific test data
- State-specific business rules
- State-specific API endpoints

7.2 Supported States

State	Abbreviation	Key Features
Pennsylvania	PA	Full SSAP support
New Jersey	NJ	GetCoveredNJ platform
Nevada	NV	State-specific enrollment
Idaho	ID	Unique application flow
Georgia	GA	Carrier data integration
Virginia	VA	State-specific rules
Minnesota	MN	MN-specific features
Illinois	IL	IL marketplace
New Mexico	NM	NM-specific configurations

7.3 State Resolution Mechanism

Selector Resolution Flow

1. Check for state-specific selector (e.g., selector.PA)
2. If not found, check for common selector (selector.common)
3. If not found, use string selector as-is

Content Resolution Flow

1. Check UI language (English/Spanish)
2. If Spanish: Check spanish[state] → spanish.common
3. If English: Check [state] → common

7.4 State Configuration

State is configured via environment variable:

```
state=PA # Sets current state to Pennsylvania
```

Accessed globally:

```
global.state = process.env.state;
```

8. Selector & Content Management

8.1 Selector Management System

Architecture

Selectors are stored in JSON files organized by:

1. **Module** (SSAP, MemberPortal, Agency, etc.)
2. **State** (common vs. state-specific)

Selector File Structure

Location: resources/selectors/common/SSAP/LoginPage.json

```
{
  "emailField": {
    "common": "[data-testid='email']",
    "PA": "#pa-email-field",
    "NJ": "#nj-email-input"
  },
  "passwordField": {
    "common": "[data-testid='password']"
  },
  "submitButton": {
    "common": "button[type='submit']",
    "PA": "#pa-submit-btn",
    "spanish": {
      "common": "button[type='submit']",
      "PA": "#pa-enviar-btn"
    }
  }
}
```

Selector Resolution Logic

```

getSelector(selector) {
  if (typeof selector !== "string") {
    // Check for Spanish language
    if (global.uiLanguage === "ES") {
      if (selector.spanish && selector.spanish[state]) {
        return selector.spanish[state];
      } else if (selector.spanish && selector.spanish.common) {
        return selector.spanish.common;
      }
    }

    // Check for state-specific selector
    if (selector[state.toUpperCase()]) {
      return selector[state];
    }

    // Fallback to common selector
    if (selector.common) {
      return selector.common;
    }
  }

  // Return string selector as-is
  return selector;
}

```

Usage in Page Models

```

const selectors = require('../resources/selectors/common/SSAP/LoginPage.json');

class LoginPage {
  constructor() {
    this.emailField = selectors.emailField;
    this.submitButton = selectors.submitButton;
  }

  async login(email) {
    // Browser wrapper resolves state-specific selector
    const emailSelector = browser.getSelector(this.emailField);
    await browser.setValueInTextField(emailSelector, email);

    const submitSelector = browser.getSelector(this.submitButton);
    await browser.click(submitSelector);
  }
}

```

8.2 Content Management System

Architecture

Content (UI text, messages, labels) is stored in JavaScript modules.

Content File Structure

Location: resources/content/common/SSAP/LoginPage.content.js

```
module.exports = {
  "loginHeader": {
    "common": "Sign In to Your Account",
    "PA": "Welcome to Pennie",
    "NJ": "GetCoveredNJ Login",
    "spanish": {
      "common": "Inicie sesión en su cuenta",
      "PA": "Bienvenido a Pennie",
      "NJ": "Inicio de sesión GetCoveredNJ"
    }
  },
  "errorMessage": {
    "common": "Invalid credentials",
    "PA": "Please check your username and password",
    "spanish": {
      "common": "Credenciales inválidas"
    }
  }
};
```

Content Resolution Logic

```
getContent(content) {
  if (typeof content !== "string") {
    // Check for Spanish language
    if (global.uiLanguage === "ES") {
      if (content.spanish && content.spanish[state]) {
        return content.spanish[state];
      } else if (content.spanish && content.spanish.common) {
        return content.spanish.common;
      }
    }

    // Check for state-specific content
    if (content[state.toUpperCase()]) {
      return content[state];
    }

    // Fallback to common content
    if (content.common) {
      return content.common;
    }
  }

  // Return string content as-is
}
```

```
    return content;
}
```

Usage in Page Models

```
const content = require('../../resources/content/common/SSAP/LoginPage.content.js');

class LoginPage {
  constructor() {
    this.expectedHeader = content.loginHeader;
  }

  async verifyPage() {
    const expectedText = browser.getContent(this.expectedHeader);
    const actualText = await browser.getText(this.pageHeader);
    expect(actualText).toContain(expectedText);
  }
}
```

8.3 Benefits of Centralized Management

1. **Maintainability:** Single source of truth for selectors and content
 2. **Consistency:** Uniform handling across all tests
 3. **State Support:** Easy addition of new states
 4. **Language Support:** Built-in internationalization
 5. **Version Control:** Track changes to UI elements
 6. **Reusability:** Share selectors across multiple page models
-

9. Test Execution Flow

9.1 Execution Architecture

```
npm test
  ↓
environment.env (loads environment variables)
  ↓
cucumber.js (configures test runner)
  ↓
Hooks.js (BeforeAll - browser setup)
  ↓
Feature Files (scenarios to execute)
  ↓
Step Definitions (implement test logic)
  ↓
Page Models (perform UI interactions)
  ↓
Browser Wrapper (abstracts Playwright)
  ↓
Playwright API (browser automation)
  ↓
```

```
Allure Reporter (collects test results)
  ↓
allure-results/ (raw test data)
  ↓
npm run allure:generate (generates HTML report)
  ↓
allure-report/ (HTML report)
```

9.2 Test Execution Steps

Step 1: Environment Setup

```
# Load environment variables
source environment.env
# OR
env-cmd -f environment.env
```

Step 2: Test Execution

```
# Run all tests
npm test

# Run specific tags
npm test -- --tags "@smoke"

# Run specific state
npm test -- --tags "@PA"

# Run with specific browser
BROWSER=firefox npm test
```

Step 3: Report Generation

```
# Generate Allure report
npm run allure:generate

# Open report in browser
npm run allure:open
```

9.3 Parallel Execution

The framework supports parallel test execution:

```
// cucumber.js
--parallel 16 // Run 16 scenarios in parallel
```

Benefits:

- Faster test execution

- Better resource utilization
- Reduced overall test time

Considerations:

- Tests must be independent
- Proper test data isolation required
- Database cleanup between tests

9.4 Retry Mechanism

Failed tests can be automatically retried:

```
// cucumber.js
--retry 1 // Retry failed tests once
```

Configuration:

- Global retry in `cucumber.js`
- Per-scenario retry via tags
- CI-specific retry in `playwright.config.js`

10. Data Management

10.1 Test Data Organization

Test data is organized by:

1. **State:** State-specific data folders
2. **Module:** Feature-specific data files
3. **Type:** JSON, CSV, XML formats

10.2 Data File Structure

```
resources/data/
├─ Common/                # Shared data across states
│   └─ users.json          # Common user data
│   └─ addresses.json      # Common address data
├─ PA/                     # Pennsylvania-specific data
│   └─ SSAP/               # SSAP test data
│       └─ MemberPortal/   # Member portal data
├─ NJ/                     # New Jersey-specific data
└─ [other states]/        # Additional state data
```

10.3 Data File Format

Example: `resources/data/PA/SSAP/householdData.json`

```
{
  "individual": {
    "firstName": "John",
    "lastName": "Doe",
    "dateOfBirth": "01/01/1990",
```

```
    "email": "john.doe@test.com"
  },
  "family": {
    "primaryApplicant": {
      "firstName": "John",
      "lastName": "Doe"
    },
    "spouse": {
      "firstName": "Jane",
      "lastName": "Doe"
    },
    "children": [
      {
        "firstName": "Child1",
        "lastName": "Doe"
      }
    ]
  }
}
```

10.4 Data Loading Utilities

```
const dataUtil = require('../common.utils/DataUtil');

// Load test data
const testData = await dataUtil.getTestData('householdData', state);

// Access data
const firstName = testData.individual.firstName;
```

10.5 Dynamic Data Generation

The framework includes utilities for generating dynamic test data:

```
const random = require('../common.utils/RandomDataGenerator');

// Generate random user
const userData = await random.generateUser(state);

// Generate random household
const householdData = await random.generateHousehold(memberCount);
```

10.6 Data Validation

Test data can be validated against database:

```
const dbHelper = require('../common.utils/DatabaseHelper');
```

```
// Validate user exists in database
const userExists = await dbHelper.validateUser(userId);
```

11. Reporting & Analytics

11.1 Allure Reporting

The framework uses Allure for comprehensive test reporting.

Report Features

1. Test Overview

- Total tests executed
- Pass/Fail statistics
- Execution duration
- Test trends

2. Detailed Test Results

- Step-by-step execution
- Screenshots on failures
- Console logs
- Test data and parameters

3. Test Categories

- Grouped by features (@SSAP, @MemberPortal)
- Grouped by states (@PA, @NJ)
- Grouped by severity levels

4. Historical Trends

- Pass/fail trends over time
- Performance metrics
- Flaky test identification

11.2 Report Generation Process

Step 1: Test Execution

```
npm test # Generates allure-results/
```

Step 2: Generate HTML Report

```
npm run allure:generate # Generates allure-report/
```

Step 3: View Report

```
npm run allure:open # Opens report in browser
```

11.3 Screenshot Capture

Screenshots are automatically captured:

- On test failures
- On assertion failures
- On explicit screenshot calls

Implementation:

```
// In Hooks.js - AfterStep
if (scenario.result.status === 'FAILED') {
  let screenshot = await browser.takeScreenshot();
  this.attach(screenshot, "image/png");
}
```

11.4 Logging

The framework includes comprehensive logging:

```
const logger = require('../common/utils/LoggerUtil');

logger.log("Info message");
logger.error("Error message");
logger.warn("Warning message");
```

Logs are attached to Allure reports for debugging.

11.5 Custom Report Attachments

Additional data can be attached to reports:

```
// Attach JSON data
this.attach(JSON.stringify(testData), 'application/json');

// Attach text file
this.attach(logContent, 'text/plain');

// Attach screenshot
this.attach(screenshot, 'image/png');
```

12. Best Practices & Guidelines

12.1 Test Design Principles

1. Independent Tests

- Each test should run independently
- No dependencies between tests
- Clean up test data after execution

2. Data-Driven Testing

- Use external data files
- Parameterize test scenarios
- Avoid hardcoded values

3. Modular Design

- Break complex scenarios into smaller steps
- Reuse common step definitions
- Create utility functions for repeated operations

12.2 Coding Standards

Naming Conventions

- **Feature files:** `FeatureName.feature` (PascalCase)
- **Step definitions:** `FeatureNameStepDef.js` (PascalCase + StepDef)
- **Page models:** `PageNamePage.js` (PascalCase + Page)
- **Variables:** `camelCase`
- **Constants:** `UPPER_SNAKE_CASE`

Code Structure

```
// ✅ Good: Well-structured page model
class LoginPage {
  constructor() {
    const selectors = require('../resources/selectors/common/SSAP/LoginPage.json');
    this.emailField = selectors.emailField;
  }

  async login(email, password) {
    await browser.waitForElementToDisplay(this.emailField);
    await browser.setValueInTextField(this.emailField, email);
    // ... rest of implementation
  }
}

// ❌ Bad: Hardcoded selectors
class LoginPage {
  async login(email) {
    await browser.setValueInTextField("#email", email); // Hardcoded selector
  }
}
```

12.3 Element Interaction Best Practices

Always Wait for Elements

```
// ✅ Good: Wait before interaction
await browser.waitForElementToDisplay(selector);
await browser.click(selector);

// ❌ Bad: Immediate interaction
await browser.click(selector); // May fail if element not ready
```

Use State-Aware Selectors

```
// ✅ Good: Use selector resolution
const selector = browser.getSelector(this.submitButton);
await browser.click(selector);

// ❌ Bad: Hardcode state-specific logic
if (state === 'PA') {
  await browser.click('#pa-submit');
} else {
  await browser.click('#submit');
}
```

12.4 Error Handling

Implement Proper Error Handling

```
// ✅ Good: Comprehensive error handling
async performAction() {
  try {
    await browser.click(selector);
    logger.log("Action completed successfully");
  } catch (error) {
    logger.error(`Action failed: ${error.message}`);
    await browser.takeScreenshot();
    throw error;
  }
}

// ❌ Bad: No error handling
async performAction() {
  await browser.click(selector); // May fail silently
}
```

12.5 Test Data Management

Use External Data Files

```
// ✅ Good: Load from data file
const testData = await dataUtil.getTestData('householdData', state);
await pageModel.fillForm(testData);

// ❌ Bad: Hardcoded data
await pageModel.fillForm({
  firstName: "John",
  lastName: "Doe"
});
```

12.6 Documentation

Document Complex Logic

```
/**
 * Processes SSAP application data and submits to backend
 * @param {Object} data - Application data object
 * @param {string} data.applicationType - Type of application (individual/family)
 * @param {Array} data.household - Array of household member objects
 * @param {string} state - State abbreviation (PA, NJ, etc.)
 * @returns {Promise<string>} - Application ID if successful
 */
async processApplication(data, state) {
  // Implementation
}
```

13. Troubleshooting Guide

13.1 Common Issues

Issue: Element Not Found

Symptoms: Element not found or Timeout waiting for element

Solutions:

1. Verify selector accuracy in browser DevTools
2. Check if element is in iframe
3. Ensure proper wait conditions
4. Verify state-specific selector configuration

```
// Debug element location
const isVisible = await browser.isElementDisplayed(selector);
const count = await browser.findNumberOfElements(selector);
console.log(`Element visible: ${isVisible}, Count: ${count}`);
```

Issue: State-Specific Selector Not Working

Symptoms: Test passes in one state but fails in another

Solutions:

1. Verify selector file exists for the state
2. Check selector resolution logic
3. Verify state environment variable is set correctly

```
// Verify state configuration
console.log(`Current state: ${global.state}`);

// Verify selector structure
console.log(`Selector structure:`, JSON.stringify(selectors.submitButton, null, 2));
```

Issue: Test Data Not Loading

Symptoms: Cannot read property of undefined

Solutions:

1. Verify data file exists
2. Check data file structure
3. Verify state-specific data path

```
// Debug data loading
const testData = await dataUtil.getTestData('fileName', state);
console.log(`Loaded test data:`, JSON.stringify(testData, null, 2));
```

Issue: Browser Crashes or Timeouts

Symptoms: Browser crashes or tests timeout

Solutions:

1. Increase timeout values
2. Check browser memory usage
3. Verify network connectivity
4. Review browser launch arguments

```
// Increase timeouts in playwright.config.js
module.exports = defineConfig({
  timeout: 120000,
  use: {
    actionTimeout: 30000,
    navigationTimeout: 60000
  }
});
```

13.2 Debugging Techniques

Enable Debug Logging

```
# Enable Playwright debug logs
DEBUG=pw:api npm test

# Enable Cucumber debug logs
DEBUG=cucumber:* npm test
```

Run Tests in Non-Headless Mode

```
# Run with visible browser
HEADLESS=false npm test
```

Use Browser DevTools

```
// Pause execution for manual inspection
await browser.pause(30000); // 30 second pause
```

13.3 Performance Issues

Slow Test Execution

Solutions:

1. Optimize wait conditions
2. Reduce unnecessary browser interactions
3. Use API calls for data setup
4. Implement parallel execution

Memory Consumption

Solutions:

1. Ensure proper cleanup in After hooks
2. Close unused browser contexts
3. Limit concurrent browser instances

14. Advanced Topics

14.1 API Testing Integration

The framework supports API testing alongside UI testing:

```
const axios = require('axios');

// Make API call
const response = await axios.post(apiUrl, payload);

// Validate response
expect(response.status).toEqual(200);
expect(response.data.applicationId).to.exist;

// Store in global data store
global.updateDataJson.api.response = response.data;
```

14.2 Database Integration

Direct database queries for validation:

```
const dbHelper = require('../common.utils/DatabaseHelper');

// Query database
const result = await dbHelper.executeQuery(
  'SELECT * FROM applications WHERE id = ?',
  [applicationId]
);
```

```
// Validate data
expect(result[0].status).toEqual('SUBMITTED');
```

14.3 Custom Assertions

The framework includes custom assertion utilities:

```
const assert = require('../..base/Assert');

// Soft assertions (continue on failure)
await assert.softAssertTrue(condition, "Error message");

// Hard assertions (stop on failure)
await assert.assertTrue(condition, "Error message");
```

14.4 Visual Regression Testing

The framework supports visual regression testing:

```
// Compare screenshots
await browser.takeScreenshot('baseline.png');
await browser.compareScreenshots('baseline.png', 'current.png');
```

14.5 Accessibility Testing

Accessibility testing using axe-playwright:

```
const { injectAxe, checkA11y } = require('axe-playwright');

// Inject axe
await injectAxe(page);

// Check accessibility
await checkA11y(page);
```

Conclusion

This framework provides a comprehensive, maintainable, and scalable solution for UI automation testing of Health Insurance Exchange platforms. By following the patterns and practices outlined in this documentation, teams can:

1. **Maintain** tests efficiently with centralized selector and content management
2. **Scale** testing across multiple states and features
3. **Collaborate** effectively using BDD approach
4. **Report** comprehensively with Allure integration
5. **Debug** efficiently with detailed logging and screenshots

Key Takeaways

- **Centralized Management:** Selectors and content are managed centrally, reducing maintenance overhead
- **State-Aware:** Built-in support for multi-state testing with automatic resolution

- **BDD Approach:** Tests are readable and maintainable using Gherkin syntax
- **Comprehensive Reporting:** Allure provides detailed test analytics and trends
- **Extensible:** Framework can be easily extended with new features and states

Next Steps

1. Review existing test scenarios to understand patterns
2. Practice writing new tests following the guidelines
3. Explore advanced features like API testing and database integration
4. Contribute improvements to the framework
5. Share knowledge with team members

Document Version: 1.0

Last Updated: December 2024

Maintained By: Automation Team

For Questions: Refer to team documentation or contact framework maintainers